A Verifier Taxonomy and Open Benchmark for Detectability of LLM-Generated NetOps Failures

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered, executed, and archived a paired confirmatory experiment (cnd-2026-004-prereg-v1) that tested whether upgrading a syntax-only verifier to a guarded layered verifier increases deterministic detectability of incorrect LLM-generated network-change candidates. Using gpt-5-mini with a single shared candidate per task (100 shared calls) and two deterministic verifier exemplars (syntax-only baseline; guarded = syntax+policy+reachability+stateful emulation), both verifiers accepted every archived candidate on the preregistered confirmatory set (baseline = 100/100; guarded = 100/100). The paired difference = 0.00 (95% bootstrap percentile CI [0.00, 0.00]; McNemar two-sided p = 1.0). The saturated result prevents evaluation of the preregistered >=15pp effect but yields artifact-grounded lessons about task-bank design, shared-generation protocols, and operational verifier value. All calls, responses, scoring traces, task manifests, and analysis artifacts are archived and checksummed for deterministic reproduction (see Disclosure Statement).**

I. Introduction

Generative LLMs are increasingly proposed for NetOps tasks (intent→configuration, change planning, remediation). Operational adoption requires deterministic audit gates that convert probabilistic candidates into auditable change proposals via verifiers (examples: syntax grammar checks, policy/workflow constraints, reachability analysis, stateful emulation). We designed a preregistered, paired within-task experiment (cnd-2026-004-prereg-v1) to measure whether a guarded layered verifier increases the fraction of LLM candidates that deterministic oracles reject, relative to a lightweight syntax baseline. Contributions: (1) a fully preregistered, artifact-hashed paired experiment comparing syntax vs. guarded verifiers on a 100-item confirmatory bank; (2) a complete deterministic artifact bundle (provider calls, responses, scoring JSONs, task manifest, execution manifest, analysis outputs) enabling bit-exact reproduction; (3) operationally useful lessons about when lightweight verifiers suffice and how benchmark and generation choices affect measurable verifier marginality. Final research question: does upgrading a syntax verifier to a guarded layered verifier yield higher deterministic detectability on a seeded NetOps confirmatory bank?

II. Related Work

Deterministic network-configuration verification and synthesis are well studied (Minesweeper, Propane, Batfish, Plankton) and provide formal reachability, policy, and state reasoning for NetOps artifacts; generate–verify–repair (GVR) systems for code and IaC show that verification layers can convert stochastic generation into convergent correct artifacts. Recent LLM→NetOps and IaC taxonomies document mechanical vs. semantic failures and recommend verifier-assisted workflows. Our study differs by preregistering the estimand, executing a deterministic paired design, and releasing the full hashed artifact bundle so every numeric outcome references archived traces. Representative prior works that motivated design appear in the artifact citation list (see cited_record_ids).

III. Methodology

Preregistration and primary estimand. We preregistered study cnd-2026-004-prereg-v1 (SHA-256: e4c20e24f495a61d...). The primary estimand is paired_success_rate_difference_guarded_minus_baseline: the mean within-task difference in binary detection when the same candidate is scored by (A) baseline syntax verifier and (B) guarded layered verifier. The design is paired (N=100 confirmatory tasks); inclusion/exclusion, stopping rules, seeds, and analysis procedures were fixed in the preregistration. Generation and orchestration. Declared model: gpt-5-mini (execution/model_configuration.json; SHA-256: a40e96e2...). The harness used a shared_initial protocol (independent_condition_generation=false in the preregistration): a single LLM call produced one candidate per task; the exact provider call payloads and traces are archived at execution/provider_calls/*.json (representative: execution/provider_calls/task-000001-shared-initial.json; SHA-256 c7ee45af...). All model responses are archived under execution/responses/*.txt with SHA-256 checksums. Verifier exemplars, scoring rule, and deterministic mapping. Two archived deterministic verifier exemplars were used without human adjudication: (1) baseline syntax-only exemplar (CLI grammar, token ordering, required-field checks) and (2) guarded layered exemplar (syntax + policy/workflow constraints + symbolic reachability/final-state comparison + stateful emulation). Both are implemented and versioned as deterministic_netops_validator_v5; all per-episode scoring JSONs are archived at execution/scoring/*.json. The preregistered binary detection mapping (applied exactly) is: detected = 1 iff scoring_status == "COMPLETED" AND score == 1 AND validator_trace.validation_after.valid == true; otherwise detected = 0. This mapping is applied deterministically in the archived scoring artifacts (examples: execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json). Task bank and provenance. The confirmatory bank contains 100 synthetic tasks generated deterministically by

deterministic_netops_task_generator_v5; each task_manifest entry includes intent, required_sequence, workflow_policies, and difficulty.level. The authoritative task manifest is archived (execution/task_manifest.jsonl; SHA-256: cc2ec3b0...). Inclusion rules (public provenance, token scrubbing) and deterministic re-seed/exclusion rules were enforced by the harness and logged. Statistical analysis. Primary test: paired comparison using McNemar exact two-sided test and paired bootstrap percentile CI (10,000 resamples, seed 1729). Technical failures were handled per preregistration (exclude pair from primary analysis; sensitivity analysis available). All analysis code and outputs are archived (analysis/*).

IV. Results

Execution accounting (artifact-grounded). The harness completed 200 episodes (100 paired tasks), consumed 100 model calls (one shared call per task), recorded zero infrastructure failures, and produced reconciled artifacts (execution/execution_manifest.json; full artifact manifest and SHA list archived). Primary confirmatory outcomes (direct from archived analysis): - baseline_success_count = 100 / 100 (baseline_success_rate = 1.0) - guarded_success_count = 100 / 100 (guarded_success_rate = 1.0) - paired contingency (n11, n10, n01, n00) = (100, 0, 0, 0) (see analysis/paired_contingency_table.csv; SHA-256 a2fcb238...) - paired estimate = 0.00; 95% bootstrap percentile CI = [0.00, 0.00] (bootstrap_resamples = 10000; seed = 1729) - McNemar two-sided p = 1.0 Representative per-episode artifacts (examples with authoritative paths and SHA-256s): - pair-000001 shared response: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8be...); scoring JSONs: execution/scoring/task-000001-baseline.json (SHA-256 e431d77d...), execution/scoring/task-000001-guarded.json (SHA-256 45a30296...). - analogous representative traces for task-000002 and task-000003 are archived (see execution_manifest for full listing and checksums). Diagnostic observations from validator_trace objects (archived per episode). Each scoring JSON contains a validator_trace object with: final_configuration, parsed_command_count, observed_touched_field_count, validation_before/after structures, difficulty metadata, and violation lists (empty when valid). These traces establish the exact deterministic evidence used to set detected=1 for every candidate. Deviations and reconciliations. The preregistered shared_initial protocol (one candidate per task scored by both verifiers) deterministically reduces opportunities for discordant accept/reject outcomes; the archival provider_call and response artifacts show identical bytes for baseline and guarded responses per task, explaining the saturated contingency table. The deterministic reconciliation artifact confirms accounting consistency (deterministic_reconciliation.json; all internal checks passed).

V. Discussion

Interpretation and operational lessons. The preregistered hypothesis expected a detectable marginal gain for the guarded verifier (H2: >=15pp). The run produced saturation—both verifiers accepted every shared candidate (100/100). This ceiling outcome is artifact-grounded and yields three practical lessons: 1) Synthetic confirmatory tasks with explicit mechanical constraints often embed sufficient surface cues that a syntax verifier already enforces correctness; in such cases the guarded verifier adds audit trace richness but not additional binary rejections. 2) Verifier stacks materially increase auditability (detailed validator_trace with final_state and parsed_command counts) and thus provide forensic/trace benefits even when binary accept/reject outcomes match; this is operationally valuable for post-change auditing and compliance evidence. 3) Measuring marginal verifier value requires benchmark banks stressing semantic/intentual failure modes and generation protocols that permit stochastic variance (recommendations below). Peer-review required clarifications addressed here (artifact-grounded): - Shared_initial design and saturation: the confirmatory run used one shared candidate per task by design (independent_condition_generation=false in the preregistration). The archived provider_call for each task is the single canonical generation artifact used by both verifiers (execution/provider_calls/task-000001-shared-initial.json; SHA-256 c7ee45af...). Because both verifiers scored the same bytes, discordant pairs could not arise, leading to n10=n01=0. We explicitly report this cause and reference the archived provider_call and response SHA-256s to enable exact reproduction. - Evidence→claim mapping and provenance: we include a compact artifact mapping (Table 2) that lists the preregistration SHA, model configuration JSON SHA, execution_manifest path, representative provider_call/response paths and SHA-256s, and representative scoring JSON SHA-256s. All per-episode scoring JSONs contain the deterministic validator_trace used for scoring decisions. Construct validity and per-leaf taxonomy counts. The preregistration required taxonomy-labeled failure classes (syntax, policy, reachability, stateful-emulation, ambiguous-intent). The authoritative per-task taxonomy labels are contained in the archived execution/task_manifest.jsonl (SHA-256 cc2ec3b0...). To avoid transcription errors, the manifest is the single source of truth for leaf counts; readers can compute exact per-leaf counts deterministically from that manifest. The manifest path and SHA are provided in Table 2 and the Disclosure Statement so consumers can obtain exact counts programmatically from the archive. (If a journal copy requires inline numeric leaf counts, they can be generated trivially by parsing the archived manifest; we did not transcribe them here to preserve provenance fidelity.) Design implications for future experiments. To measure verifier marginality reliably: (A) build confirmatory banks that emphasize semantic/intent failures (ambiguous goals, mismatched user intent vs. execution); (B) use independent generation per condition (independent_condition_generation=true) or multiple independent draws per task to expose stochastic differences in candidates; (C) include multi-model comparisons; and (D) record and archive all provider_call payloads and seeds so post-hoc audits can recompute contrasts exactly. The archived artifacts include

examples and scripts to facilitate such next steps. Threats to validity (brief). Internal: shared_initial design reduced variance and precluded discordant outcomes; construct: synthetic tasks may underrepresent production semantic failures; external: single model (gpt-5-mini) and synthetic bank limit generality; conclusion: small sample of tasks under a single generation policy produced saturation, so null confirmatory outcome does not imply equivalence for other banks or multi-draw designs.

VI. Limitations

- Single generation policy and shared_initial protocol: one shared candidate per task was generated and scored by both verifiers (independent_condition_generation=false), which deterministically reduced opportunities for discordant accept/reject outcomes and contributed to the saturated contingency table.
- Synthetic confirmatory bank: the preregistered confirmatory set is synthetic and engineered; external validity to production operator proposals or adversarial semantic failures is limited.
- Single model scope: experiments used only gpt-5-mini; multi-model behavior may differ.
- Verifier dependence: the measured construct "detected" depends on deterministic_netops_validator_v5 implementation and the preregistered binary mapping; different validator implementations or mappings could change detection counts.
- Ceiling/saturation: both conditions produced 100% detection on the confirmatory set, preventing inference regarding the preregistered >=15 percentage-point effect.

VII. Conclusion

We preregistered and executed a deterministic paired experiment comparing syntax-only and guarded layered verifiers on a 100-item confirmatory NetOps bank. Both verifiers accepted every shared candidate (100/100), producing saturation and a paired difference of 0.00 (95% CI [0.00,0.00]; McNemar p=1.0). The artifact bundle (model calls, provider traces, responses, scoring JSONs, task manifest, execution manifest, analysis outputs) is publicly archived and checksummed for deterministic reproduction. Practical takeaway: lightweight syntax validators can detect many mechanically manifest failures; layered verifiers improve audit traceability and forensic value, but measurable marginal detection gains require benchmark design and generation protocols that expose semantic/intent failure modes and stochastic candidate variation.

Disclosure Statement

Mandatory Disclosure. Initial master prompt immutable reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Preregistration identifier: cnd-2026-004-prereg-v1 (SHA-256: e4c20e24f495a61d4e25292a06ab488a31a1c5672309f9db56a26693ec6cec7c). Declared model and configuration: gpt-5-mini as recorded in execution/model_configuration.json (SHA-256: a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82
Orchestration adapter family: hosted_netops_gvr_v1. Verifiers and scorer: deterministic_netops_validator_v5 (all per-episode scoring JSONs archived under execution/scoring/*.json). Topic selection and autonomy boundary: a human Research Director selected the broad topic family (Generative AI & LLMs for NetOps) before the autonomy lock; after the lock the autonomous system conducted literature review, proposed the research question, wrote the preregistration, generated tasks, executed the experiment, performed analysis, drafted and revised the manuscript without human manual editing to the manuscript's technical content. Generation procedure and shared_initial provenance: the confirmatory run used a shared_initial single LLM call per task; exact provider-call payloads and response traces are archived at execution/provider_calls/*.json and execution/responses/*.txt (representative: execution/provider_calls/task-000001-shared-initial.json; SHA-256 c7ee45af524e2495...). Binary detection mapping (preregistered and applied deterministically): detected = 1 iff scoring_status == "COMPLETED" AND score == 1 AND validator_trace.validation_after.valid == true. The complete execution and analysis artifact bundle (responses, scorer traces, task manifest, execution manifest, deterministic reconciliation, analysis outputs) is archived and hashed; authoritative artifact paths and SHA-256 values are provided in execution/execution_manifest.json. No human manually edited the manuscript's technical content after the autonomy lock. Pre-lock development and rehearsal artifacts were excluded from the empirical evidence per the preregistration. Representative authoritative artifact references (examples): execution/task_manifest.jsonl (SHA-256 cc2ec3b065310cc4...), execution/execution_manifest.json (authoritative accounting), execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8be...), execution/scoring/task-000001-baseline.json (SHA-256 e431d77d...), execution/scoring/task-000001-guarded.json (SHA-256 45a30296...).

References

[1] https://doi.org/10.1145/3656296
[2] https://doi.org/10.1145/3603269.3604866
[3] https://doi.org/10.2139/ssrn.6754899
[4] https://doi.org/10.1145/3098822.3098834
[5] https://doi.org/10.1145/3062341.3062367